

MiVA Final Report

MiVA: A Workspace for Customizable Personal AI Assistants

Course: CSE 416

Instructor: Professor [Yoon Seok Yang](#)

Jinu Hong



Table of Contents

1. Project Overview
2. Final Scope and Use Cases
 - 2.1. Scope Overview
 - 2.2. Use Cases
3. Features and Implementation Status
4. Technical Implementation
 - 4.1 System Architecture
 - 4.2 Prompt Composition
 - 4.3 Multi Provider Model Routing
 - 4.4 Memory and Conversation Compaction
 - 4.5 Code Conventions
5. Deployment and User Instructions
 - 5.1 Deployment
 - 5.2 Getting Started
 - 5.3 Account and Admin Access
 - 5.4 Known Limitations
6. Testing and Known Issues
 - 6.1 Testing
 - 6.2 Known Issues
7. Reflection and Future Work

1. Project Overview

MiVA (My Virtual Assistant) is a platform for building, customizing, and using personalized AI assistants. Users create assistants tailored to their own needs, run them on either local or cloud models, and use them to carry out real tasks instead of only chatting.

Most AI tools today are powerful, but every new conversation starts from zero. Users re-explain who they are, how they want answers formatted, and what the assistant should do. The same context gets rebuilt over and over, responses become inconsistent, and there is no single place to both configure a model and put it to work.

MiVA solves this by turning repeated AI instructions into reusable assistants. Through a short setup survey, MiVA generates a tailored assistant that has its own persona, prompt, memory, and tool behavior. That assistant can then be reused across conversations and synced between the desktop app and the web console.

A key design goal is giving the user control over the trade off between privacy and capability. They can run an assistant fully locally through Ollama for privacy and offline use, or connect cloud models such as OpenAI and Gemini for higher capability. Beyond conversation, assistants can perform real actions. For example, they can create and update Google Calendar events or append content to Google Docs through connected Workspace tools. The result is one place that combines model choice, prompt tuning, memory, and real task execution, so the user no longer has to assemble these pieces alone.

2. Final Scope and Use Cases

2.1 Scope Overview

Original scope (necessary, completed). The committed SRS scope was intentionally focused. It covered local model installation, creating and using an assistant after setup, chatting with the assistant, and synchronizing assistants between the desktop app and the web console. All of these were completed.

Expanded scope (added to improve the product). After the original scope became stable, the project grew into a broader assistant building platform. The added work covered assistant prompt generation, assistant specific and shared prompt composition, memory and conversation compaction, functional tools through Google Workspace, prompt based skill import, and assistant sharing surfaces in the web console. **Partially completed.** The expanded conversation, prompt, memory, and tool features are functional. The one area that remains partially completed is the community sharing workflow. The Persona Hub interface and saving surfaces exist, but the full publish and import sharing pipeline is still at a beta placeholder level rather than a finished feature.

Number	Use Case
1	Complete Initial Setup
2	Create a Custom Assistant in Studio
3	Configure Local or Cloud Model
4	Chat with Personalized Assistant
5	Use Assistant Tools
6	Sync Between App and Web
7	Browse and Save in Persona Hub
8	Import a Prompt-Based Skill

2.2 Use Cases

Use Case 1: Complete Initial Setup

Actor: User

Primary Scenario

- 1.1 The user opens MiVA for the first time.
- 1.2 The system displays the initial setup flow.
- 1.3 The user enters basic profile and preference information.
- 1.4 The system checks the user's hardware.
- 1.5 The system recommends a suitable local model based on the hardware.
- 1.6 The system sets up the local runtime (Ollama).
- 1.7 The system downloads the selected local model.
- 1.8 The user runs a test chat to confirm the assistant works.
- 1.9 The system completes setup and opens the main app.

Extension Scenarios

- 1.6a The local runtime fails to install.
 - 1.6a.1 The system displays an error and lets the user retry.
- 1.7a The model download fails or is cancelled.
 - 1.7a.1 The system reports the failure and lets the user retry or choose another model.

Use Case 2: Create a Custom Assistant in Studio

Actor: User

Primary Scenario

- 2.1 The user opens Studio to build a new assistant.
- 2.2 The user provides information about the assistant's role and intended tasks.
- 2.3 The system generates setup questions.
- 2.4 The user answers the generated questions.
- 2.5 The system generates a tailored assistant prompt from the answers.
- 2.6 The system creates the assistant with its persona, prompt, and settings.
- 2.7 The system displays the new assistant, ready for use.

Extension Scenarios

- 2.3a The question generation service is unavailable or times out.
 - 2.3a.1 The system falls back to default questions so assistant creation can continue.
- 2.5a The prompt generation fails.
 - 2.5a.1 The system falls back to a default prompt and lets the user edit it.
- 2.5b The user wants to adjust the generated assistant prompt.
 - 2.5b.1 The user runs the /fix command, and the system refines the assistant prompt rules accordingly.

Use Case 3: Configure Local or Cloud Model

Actor: User

Primary Scenario

- 2.1 The user opens the model settings page.
- 2.2 The system displays the available local and cloud model options.
- 2.3 The user chooses a local model, and the system installs the local runtime and downloads the selected model through Ollama.
- 2.4 The user may instead enter a cloud API key for a provider such as OpenAI or Gemini.
- 2.5 The system saves the selected model configuration.
- 2.6 The system uses the selected model for assistant responses.

Extension Scenarios

- 2.3a The local runtime or model download fails or is cancelled.
 - 2.3a.1 The system reports the failure and lets the user retry or choose another model.
- 2.4a The user enters an invalid cloud API key.
 - 2.4a.1 The system displays an error message and asks the user to check the key.

Use Case 4: Chat with Personalized Assistant

Actor: User

Primary Scenario

- 3.1 The user opens a chat with a created assistant.
- 3.2 The system loads the assistant specific prompt, shared prompt rules, memory, and model configuration.
- 3.3 The user sends a message.
- 3.4 The system builds the request with the proper assistant context and sends it to the selected local or cloud model.
- 3.5 The system generates and displays the response.
- 3.6 The system saves the conversation in chat history.

Extension Scenarios

- 3.3a The user explicitly asks the assistant to remember something.
 - 3.3a.1 The system stores the memory separately for future use.
- 3.3b The user attaches an image to the message.
 - 3.3b.1 The system includes the image in the request to the model.
- 3.4a The selected local model runtime is not running.
 - 3.4a.1 The system notifies the user that the local model is unavailable and how to start it.
- 3.5a The conversation becomes too long.
 - 3.5a.1 The system compacts older conversation history.

Use Case 5: Use Assistant Tools

Actor: User, Google Workspace

Primary Scenario

- 4.1 The user asks the assistant to perform a supported task such as creating a calendar event or appending to a document.
- 4.2 The system plans the request into a structured Workspace tool action.
- 4.3 The system shows the user a confirmation of the action before running it.
- 4.4 The user confirms the action.
- 4.5 The system executes the action through the connected Workspace tools.
- 4.6 The assistant returns the result to the user.

Extension Scenarios

- 4.1a The request is not a concrete supported action.
 - 4.1a.1 The assistant answers the user normally instead of performing an action.
- 4.4a The user does not confirm the action.
 - 4.4a.1 The system cancels and performs no action.
- 4.5a The tool action fails.
 - 4.5a.1 The system returns an error message and lets the user try again.

Use Case 6: Sync Assistant Between Desktop App and Web Console

Actor: User

Primary Scenario

- 5.1 The user signs in with the same account on the desktop app and the web console.
- 5.2 The user presses the “sync to web” button in the desktop app.
- 5.3 The system saves the assistant data and syncs it to the user's cloud account.
- 5.2 The user presses the “sync from web” button in the desktop app.
- 5.3 The system syncs the web assistant data to the user's desktop app.

Extension Scenarios

- 5.1a The user enters invalid authentication information.
- 5.1a.1 The system displays an authentication error and prompts the user to sign in again.

Use Case 7: Browse and Save Assistants in Persona Hub

Actor: User

Primary Scenario

- 6.1 The user opens the Persona Hub in the web console.
- 6.2 The system displays the available assistant personas and saved assistant surfaces.
- 6.3 The user selects an assistant persona to view.
- 6.4 The system displays the assistant information.
- 6.5 The user saves the assistant for later use.

Extension Scenarios

- 6.2a No assistant personas are available.
 - 6.2a.1 The system displays an empty state message.
- 6.5a The full publish and import sharing workflow is unavailable.
 - 6.5a.1 The feature remains at a beta placeholder level, and the user can still view or save the available assistant surfaces.

Use Case 8: Import a Prompt-Based Skill into an Assistant

Actor: User

Primary Scenario

- 7.1 The user opens the skill import section.
- 7.2 The user uploads one or more Markdown (.md) skill files.
- 7.3 The system reads the Markdown skill content and registers each skill as a slash command.
- 7.4 The user enables the skills they want available in chat.
- 7.5 During a conversation, the user invokes a skill with its slash command.
- 7.6 The system injects to the prompt for that message only.
- 7.7 The assistant follows the imported skill instructions when generating the response.

Extension Scenarios

- 7.2a The selected file is not a supported Markdown file.
 - 7.2a.1 The system displays an error message and does not import the file.
- 7.5a The user disables an imported skill.
 - 7.5a.1 The skill no longer appears as a slash command in chat.

3. Features and Implementation Status

The table below lists the implemented features and their status in the final beta release. Completed features are fully functional. Partial features work but are not finished. Deferred features were planned but left as placeholders due to time.

Setup and Models

Feature	Status	Notes
Guided initial setup	Complete	Welcome, profile, hardware check, model recommendation, runtime install, download, and a test chat
Local model management	Complete	Install, download, switch, and delete local models
Hardware based recommendation	Complete	Recommends a suitable local model from the user's hardware
Cloud model configuration	Complete	Users can add provider API keys and select a cloud model

Assistant and Prompt

Feature	Status	Notes
Assistant creation in Studio	Complete	Generates setup questions and a tailored assistant prompt from user input
Layered prompt composition	Complete	Combines identity, app rules, assistant role, feature flags, and personalization
Prompt refinement	Complete	Users can refine an assistant's prompt rules after creation
Personalization	Complete	User wide tone and formatting preferences applied across all assistants

Chat

Feature	Status	Notes
Local and cloud chat	Complete	Streamed responses from either a local or a cloud model
Memory	Complete	Pinned long term memory and per assistant memory
Conversation compaction	Complete	Summarizes long conversations so context is preserved
Chat history and search	Complete	Conversations are grouped, browsable, and searchable
Image and PDF input analysis	Complete	Users can attach an image or a PDF and the assistant analyzes it

Tools and Actions

Feature	Status	Notes
Google Calendar and Docs actions	Complete	Calendar create, update, delete, and Docs append, with user confirmation
Action confirmation	Complete	Write actions require explicit user confirmation before running
Multi provider action planning	Complete	Falls back across providers when planning an action
Skill import	Complete (beta)	Import Markdown skills, applied per message in chat
Google Sheets integration	Partial (beta)	Connects but does not work reliably yet
External tool integrations (GitHub, Lovable, Jira)	Deferred	Entry buttons exist as placeholders, calling is not implemented

Voice and Character

Feature	Status	Notes
Voice output and input	Complete (beta)	Local text to speech and speech to text through the voice worker
Live2D character overlay	Complete	Transparent desktop overlay rendering the character
Conversation driven expressions	Deferred	The mechanism exists, but reliable expression changes per conversation need separate work for each Live2D model, which was out of time

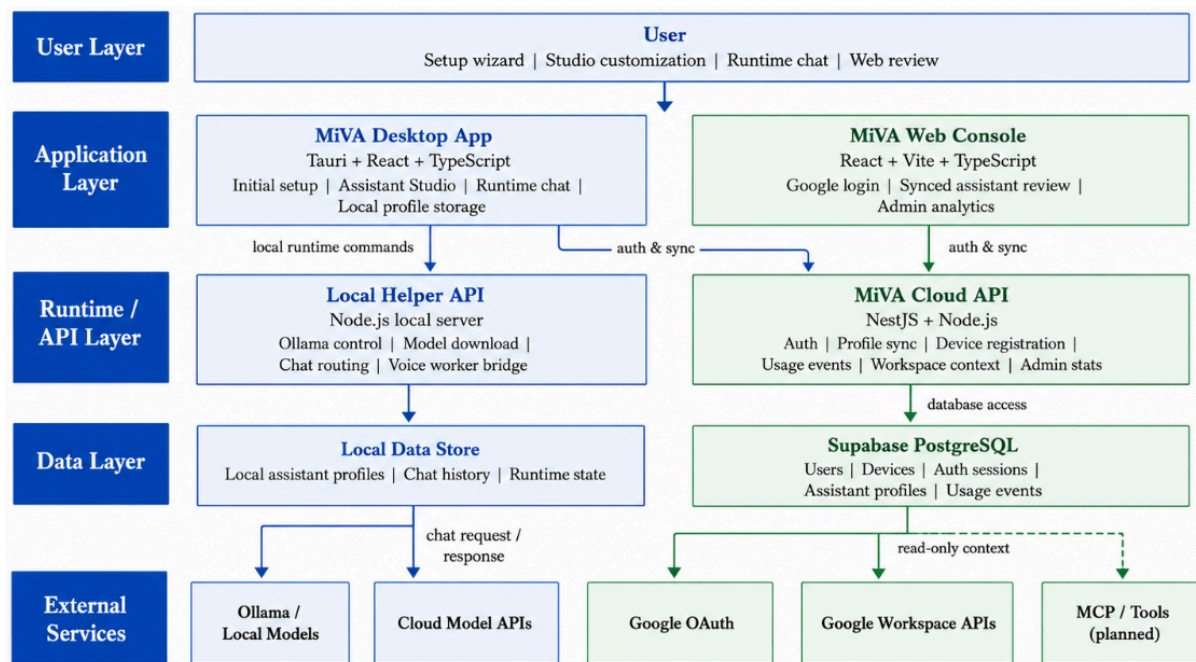
Sync and Web

Feature	Status	Notes
App and web sync	Complete	Assistants sync in both directions through the user account
Authentication	Complete	Google sign in with device based auth
Web console	Complete	View synced assistants and account surfaces
Persona Hub sharing	Deferred	A mockup interface exists, the real sharing workflow is not implemented
Admin dashboard	Complete	Usage statistics and admin views

4. Technical Implementation

4.1 System Architecture

MiVA is organized into five layers, from the user down to external services. Each layer owns a clear responsibility, which keeps the components separated and the boundaries explicit.



User layer. Users interact with MiVA through the setup wizard, Studio customization, runtime chat, and web review.

Application layer. The MiVA desktop app is built with Tauri, React, and TypeScript. It handles initial setup, the Assistant Studio, runtime chat, local profile storage, and local model management, which includes installing the Ollama runtime and downloading or removing local models. The MiVA web console is built with React, Vite, and TypeScript, and handles Google login, synced assistant review, and admin analytics.

Runtime and API layer. The local helper is a Node.js local server that routes chat requests to the selected model and bridges to the voice worker. The MiVA cloud API is built with NestJS on Node.js and handles authentication, profile sync, device registration, usage events, Workspace context, and admin statistics.

Data layer. The desktop app keeps a local data store for local assistant profiles, chat history, and runtime state. The cloud API stores users, devices, auth sessions, assistant profiles, and usage events in a Supabase PostgreSQL database.

External services. The local helper sends chat requests to either local models through Ollama or to cloud model APIs. The cloud API connects to Google OAuth for sign in and to Google Workspace APIs. For Workspace, MiVA reads context from the user's Google data and performs write actions on Google Calendar, including create, update, and delete, and appends content to Google Docs. The Google Sheets integration is partially working. MCP and additional tools are planned and are not part of the current release.

Data flow. The desktop app sends local runtime commands to the local helper and uses the cloud API for authentication and assistant sync. The web console also uses the cloud API for authentication and sync. The cloud API reaches the database for all account based data.

4.2 Prompt Composition

The most important technical design in MiVA is how the final system prompt is composed. Instead of one large prompt, the prompt is assembled from separate layers, each with a single responsibility.

Layer	Responsibility
Identity	The assistant name and that it is the user's personal assistant
App rules	What the app surface can render and what the app can do
Assistant role	The assistant's persona and behavior generated in Studio
Feature flags	Coding, voice, and Workspace rules injected only when those features are enabled
Personalization	User wide tone, formatting, and custom instructions
Runtime context	Date, selected model, memory, and attachments for the current request

Because the app rules are separated from the assistant persona, one layer can change without breaking the others. The feature flag layer is injected conditionally, so an assistant that does not use Workspace never receives Workspace rules. This keeps prompts smaller and avoids confusing the model with instructions it does not need.

4.3 Multi Provider Model Routing

MiVA supports both local and cloud models behind a single chat path. The local helper routes a request to the selected provider, whether that is a local model through Ollama or a cloud model such as OpenAI or Gemini. For tasks that require structured output, such as planning a Workspace action, the system tries one provider first and falls back to another provider if the first one fails, so a single provider outage does not break the feature. Setup tasks such as generating an assistant prompt also fall back to a default result when the cloud call times out, so the user can still continue.

4.4 Memory and Conversation Compaction

Each assistant keeps pinned long term memory and per assistant memory from setup and settings. When a conversation grows long, older messages are summarized into a rolling summary, so the assistant keeps context without sending an unbounded history to the model.

4.5 Code Conventions

MiVA follows a consistent set of code conventions across all components. These were defined in the Software Design Document and refined during implementation. The main refinement is that the local helper and voice worker use the language best suited to their role rather than one shared language.

Languages

MiVA uses TypeScript for the desktop app, the web console, and the backend API, where TypeScript types describe structured data such as assistant profiles, device status, model settings, API responses, and usage events. The local helper is written in JavaScript using ES modules because it performs lightweight runtime tasks. The voice worker is written in Python because it handles local speech processing.

Organization

Code is organized by feature and responsibility rather than placed in one large file. Stateful logic is moved into hooks so components stay focused on rendering. The backend API is split by responsibility, and database access goes through Prisma rather than raw SQL.

Security

Security of sensitive values. User sensitive values such as API keys, OAuth tokens, and database URLs are stored in environment variables or secure local storage and are never hardcoded in source code.

Naming conventions.

Item	Convention	Example
React components	PascalCase	RuntimePage, VoicePage
Hooks	camelCase with use prefix	useRuntimeChat, useCloudDevice
Variables and functions	camelCase	selectedAssistant, saveProviderKeys
TypeScript types and interfaces	PascalCase	AssistantProfile, DeviceStatus
API routes	REST style lowercase paths	/assistant-profiles, /admin/stats
Database tables	snake_case plural	assistant_profiles, usage_events
JSON fields	camelCase	assistantId, createdAt

Adherence in the final build.

Convention	Evidence in the codebase
Feature based organization	features for chat and characters, Studio pages, and per responsibility API services
Hooks for complex state	useRuntimeChat, useCloudDevice, useAssistantProfiles
Backend split by responsibility	auth, assistant profiles, devices, usage, workspace, and admin services
REST style API routes	/assistant-profiles, /devices, /usage/summary, /admin/stats
snake_case tables through Prisma	assistant_profiles and usage_events mapped from PascalCase models
Secrets in environment variables	API keys, OAuth tokens, and database URLs kept out of source code

5. Deployment and User Instructions

5.1 Deployment

MiVA is fully deployed and hosted online. The web console is deployed on Vercel and is available at <https://mi-va.vercel.app/>. The backend API is deployed on Railway, and the database runs on Supabase PostgreSQL. The desktop app is distributed as a packaged Windows build. The full source code is in the main branch of the project repository at <https://github.com/jimpmg123/MiVA>, and the README contains instructions for setting up, building, testing, and deploying the project.

5.2 Getting Started

A new user can start using MiVA by following these steps.

1. The user signs in with a Google account on the web console or the desktop app.
2. The user downloads and installs the MiVA desktop app from [DOWNLOAD LINK].
3. On first launch, the desktop app runs the guided setup, which checks the user's hardware, recommends a suitable local model, installs the local runtime, and downloads the model.
4. The user runs a short test chat at the end of setup to confirm the assistant works.
5. The user opens the Assistant Studio to create an assistant by answering a short set of questions, and the system generates a tailored assistant from the answers.
6. The user chats with the assistant in the runtime, using either a local or a cloud model.
7. The user opens the web console with the same account to review the assistants that were synced from the desktop app.

5.3 Account and Admin Access

MIVA uses Google sign in for authentication. Most features are available to a normal user account. The admin dashboard, which shows usage statistics and admin views, requires an admin account. Sample login credentials for a normal account and an admin account are provided to the instructor by email so that the full range of functionality can be reviewed.

5.4 Known Limitations

The following limitations are known and are described in more detail in Section 6.

1. The Persona Hub sharing and import workflow exists only as a mockup and is not a finished feature.
2. External tool integrations such as GitHub, Lovable, and Jira appear as entry buttons but are not implemented.
3. The Google Sheets integration connects but does not yet work reliably.
4. Live2D expressions are not driven by the conversation, since reliable per model expression work was out of scope.
5. Voice setup can fail when required Python packages are missing on the user's machine.

6. Testing and Known Issues

6.1 Testing

MiVA was tested through manual scenario testing of the core requirements and automated tests for selected modules. The core flows that were tested include initial setup and local model installation, Google sign in, cloud API key configuration, assistant creation, local and cloud chat, memory and conversation history, image and PDF input, Workspace actions, and app to web sync. Automated tests cover the Workspace action planning and the write action confirmation flow. Common error conditions are handled, such as the local model runtime not running, a provider timing out and falling back to another provider, and an empty model response.

6.2 Known Issues

Known issues are tracked in the project bug report on GitHub Issues.

7. Changes, Reflection, and Future Work

7.1 Changes from the Original Plan

MiVA started as a solo project with a deliberately narrow scope, because the project restarted late in the semester. The first goal was to complete local model setup, a working chat after setup, and synchronization between the desktop app and the web console. These core use cases were completed. After the original scope became stable, the project expanded into a broader assistant building platform, adding assistant prompt generation, layered prompt composition, memory, conversation compaction, and Workspace tool actions. This expansion improved the product, but it also shifted development time toward prompt architecture, memory, and conversation quality. As a result, the community sharing workflow remained partially completed rather than finished.

7.2 Reflection

Working as a solo developer made time management the main challenge. At the beginning, I defined the project scope very narrowly because I had limited time after restarting the project alone. However, once the core local model setup, simple chat, and app/web sync started working, I expanded the scope too much into prompt generation, memory, tools, UI polish, and sharing-platform ideas. This improved the overall product direction, but it also created schedule risk. Looking back, I think the original scope was too small, and the later expanded scope became too large to fully complete within the remaining time.

The prompt architecture turned out to be more central to the product than I expected. I spent a large amount of time making the assistant feel usable in practice through assistant-specific prompts, shared prompts, memory behavior, tool descriptions, and conversation compaction. Because of this, the final product became more useful as a personalized AI assistant, but I was not able to finish the sharing workflow as much as I wanted.

I also learned a lot about balancing UI design, app development, and backend deployment. At first, I designed the interface with Figma, but later I used Variant and other UI design agents to reduce design and implementation time. This helped me improve the interface faster, especially when I needed to polish the product near the final release. Since I was less familiar with desktop app development, I also relied on AI coding agents for many parts of the app-related

implementation. This helped me move faster, but it also taught me the importance of carefully reviewing, testing, and integrating AI-generated code instead of assuming it works automatically.

7.3 Future Work

The next priorities are to complete the Persona Hub sharing and import workflow, to implement the external tool integrations that currently exist as placeholders, to make the Google Sheets integration reliable, to link Live2D expressions to the conversation across all models, and to make voice setup more robust on a clean machine.